

Documentación Técnica: Solución del Problema de la Mochila (0/1 Knapsack)

Para este proyecto se implementa una solución para el clásico **Problema de la Mochila (0/1)** utilizando como enfoque principal la **Programación Dinámica**.

1. Enfoque de Programación Dinámica

El algoritmo presentado se puede observar que divide el problema en subproblemas más pequeños para evitar cálculos repetidos, almacenando los resultados en una matriz `dp[][]`, teniendo las siguientes características.

- **Máximo valor:** `dp[i][w]` representa el valor máximo acumulado que se puede obtener utilizando los primeros "i" elementos con una capacidad límite de mochila "cap".
- **Recurrencia:** Para cada elemento se considera las siguientes opciones:
 - **NO se incluye:** Se hereda el valor óptimo anterior: `dp[i][w] = dp[i-1][w]`
 - **O se incluye (y su peso lo permite):** Se toma su valor y se suma el óptimo del espacio restante: `currentItem.value + dp[i-1][w - currentItem.weight]`
- **Conclusión:** Se elige el máximo entre ambas opciones presentadas.

Siendo para el caso de la complejidad temporal y espacial:

- $O(N \times W)$, donde N es la cantidad de objetos y W es la capacidad de la mochila.
- $O(N \times W)$ debido al espacio requerido por la tabla de programación dinámica.

2. Estructura de Componentes

El código se divide en tres paquetes principales organizados por su responsabilidad:

Paquete	Clase	Descripción
<code>com.example</code>	<code>Main</code>	Orquestador principal. Ejecuta un caso de prueba fijo (<code>caseA</code>) y un set de experimentos analíticos (<code>executeExperiment</code>).
<code>com.example.Backpack</code>	<code>Item</code>	Modelo de datos (POJO) que define las propiedades básicas de un objeto (<code>name</code> , <code>weight</code> , <code>value</code>).
<code>com.example.Backpack</code>	<code>BPSolver</code>	Núcleo del algoritmo. Contiene la lógica DP de construcción de la matriz y el backtracking para identificar los objetos seleccionados.
<code>com.example.Benchmarking</code>	<code>Timer</code>	Utilidad métrica encargada de medir tiempos de ejecución en nanosegundos, calcular medias lógicas y realizar fases de <i>warm-up</i> .

3. Análisis del Núcleo Algorítmico (BPSolver)

El resolutor ejecuta el flujo en dos fases criticas independientes:

Fase 1: Construcción de la Tabla (createTableDP)

Genera la matriz sumando una fila y columna extra para los casos base (capacidad 0 u objetos 0). De esta manera, llenando la estructura de datos basandose en las decisiones optimas previas.

Fase 2: Reconstrucción / Backtracking (putInBackpack)

Una vez que `dp[n][cap]` guarda el valor maximo posible, el algoritmo hace un camino en reversa (desde la esquina inferior derecha hacia el origen `[0][0]`), siguiendo las siguientes consideraciones:

- Si `dp[x][y] == dp[x-1][y]`, significa que el elemento `x` **no apporto valor adicional**, por lo que se descarta y se sube de fila (`x--`).
- Si los valores difieren, el elemento **fue seleccionado**. Se añade a la lista, se resta su peso a la capacidad actual (`y -= peso`) y se sube de fila (`x--`).

Extra: (Se eligio "x e y" solo por convencion propia)

4. Modulo de Experimentacion y Benchmarking

La clase `Timer` cumple un rol fundamental para obtener métricas de tiempo reales y precisas en Java:

- **Metodo warmUp**: Ejecuta el algoritmo miles de veces antes de iniciar la medicion real. Esto fuerza al entorno de ejecución de Java (JVM) a compilar el codigo a **codigo nativo optimizado**, evitando de esta manera el posible sesgo de lentitud inicial por interpretacion (Que como curiosidad, sin este metodo el programa generaba resultados completamente inesperados).
- **Metodo getMedia**: Se encarga de tomar una media (promedio) del pedazo de codigo implementado junto a la cantidad de repeticiones que este ejecutara